Rapport Projet Cloud Computing

1- Introduction :

Ce projet a pour objectif de déployer une infrastructure cloud sur Microsoft Azure en utilisant Terraform.
L'application développée est un backend Flask permettant de gérer des fichiers, de stocker leurs métadonnées dans SQLite et de les envoyer dans Azure Blob Storage.
Le projet comprend également une machine virtuelle Ubuntu sur Azure et un déploiement automatique de l'application.

---

2. Architecture et technologies :

L'architecture du projet repose sur quatre composants principaux :

- un backend Flask ;
- une base SQLite pour les métadonnées ;
- Azure Blob Storage pour les fichiers ;
- une machine virtuelle Azure créée avec Terraform.

Terraform automatise la création de l'infrastructure, tandis qu'un script bootstrap.sh déploie automatiquement l'application sur la VM.

---

3. Développement du backend :

Le backend a été développé avec Flask.
Il permet de gérer un CRUD simple sur les fichiers et d'uploader des fichiers dans Azure Blob Storage.
Les métadonnées sont stockées localement dans une base SQLite grâce à SQLAlchemy.

Les principales routes développées sont :

- GET /
- GET /files
- POST /files
- PUT /files/<id>
- DELETE /files/<id>
- POST /upload
- GET /blobs

Test local du backend :

```
→  backend git:(main) curl http://localhost:5001/
Flask API running with Azure Blob!%
```

Test CRUD :

```
→  backend git:(main) curl http://localhost:5001/files
[{"blob_url":"https://malikblob2026efr.blob.core.windows.net/files/test.txt",
"filename":"updated_file.txt","id":1}]

→  backend git:(main) curl -X PUT http://localhost:5001/files/1 \
-H "Content-Type: application/json" \
-d '{"filename":"updated_file.txt"}'
{"message":"File updated"}
```

---

4. Mise en place du stockage Azure Blob :

Azure Blob Storage a été utilisé pour stocker les fichiers de manière persistante dans le cloud.
L'application Flask se connecte à Azure grâce au SDK azure-storage-blob et à des variables d'environnement stockées dans un fichier .env.
Lorsqu'un fichier est envoyé via l'API, il est stocké dans le conteneur files.

Liste des storage accounts ou portail Azure :

```
→  Projet_Cloud_Computing git:(main) az storage account list --output table
AccessTier    AllowBlobPublicAccess    AllowCrossTenantReplication    Creatio
nTime                      EnableHttpsTrafficOnly    Kind       Location
  MinimumTlsVersion    Name              PrimaryLocation    ProvisioningState
    ResourceGroup       StatusOfPrimary    AllowSharedKeyAccess    DefaultToO
AuthAuthentication    DnsEndpointType    EnableNfsV3    IsHnsEnabled    IsLoc
alUserEnabled    IsSftpEnabled    PublicNetworkAccess
------------  ----------------------  ----------------------------  -------
------------------------  ----------------------  ---------  -------------
  -------------------  ----------------  -----------------  -----------------
--  -----------------  ----------------  ---------------------  ----------
-------------------  -----------------  -------------  --------------  -----
---------------  ---------------  --------------------
Hot           False                    False                          2026-03
-12T14:56:46.500643+00:00  True                      StorageV2  francecentral
  TLS1_0              malikblob2026efr  francecentral      Succeeded
    flask-storage-rg    available
Hot           True                     True                           2026-03
-13T09:35:44.908050+00:00  True                      StorageV2  francecentral
  TLS1_2              malikblobtf2026   francecentral      Succeeded
    terraform-flask-rg  available          True                    False
                      Standard           False          False           True
                 False            Enabled
```

Liste des blobs :

```
→  Projet_Cloud_Computing git:(main) curl http://localhost:5001/blobs
["test.txt"]
```

---

5. Automatisation avec Terraform :

L'infrastructure a été automatisée avec Terraform.
Les fichiers provider.tf, variables.tf, main.tf et outputs.tf permettent de décrire les ressources Azure à créer.

Terraform crée automatiquement :

- un groupe de ressources
- un réseau virtuel et un sous-réseau
- une IP publique
- une interface réseau
- un groupe de sécurité réseau
- une machine virtuelle Ubuntu
- un compte de stockage Azure
- un conteneur Blob Storage

Terraform apply:

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

container_name = "files"
resource_group_name = "terraform-flask-rg"
storage_account_name = "malikblobtf2026"
vm_public_ip = "51.103.24.93"
```

Terraform destroy :

```
→  terraform git:(main) ✗ terraform destroy

Destroy complete! Resources: 10 destroyed.
```

Ressources Azure créées automatiquement avec Terraform :

Name ↑	Type	Resource Group	Location	Subscription
flask-nic	Network Interface	terraform-flask-rg	France Central	Azure for Students
flask-public-ip	Public IP address	terraform-flask-rg	France Central	Azure for Students
flask-vm	Virtual machine	terraform-flask-rg	France Central	Azure for Students
flask-vm-nsg	Network security group	terraform-flask-rg	France Central	Azure for Students
flask-vm_OsDisk_1_7bac8c3d9a1e4c1ea1edba585de9	Disk	TERRAFORM-FLASK-RG	France Central	Azure for Students
flask-vnet	Virtual network	terraform-flask-rg	France Central	Azure for Students
malikblob2026efr	Storage account	flask-storage-rg	France Central	Azure for Students
malikblobtf2026	Storage account	terraform-flask-rg	France Central	Azure for Students
NetworkWatcher_francecentral	Network Watcher	NetworkWatcherRG	France Central	Azure for Students
NetworkWatcher_swedencentral	Network Watcher	NetworkWatcherRG	Sweden Central	Azure for Students

6. Déploiement automatique sur la VM :

Le déploiement du backend sur la machine virtuelle a été automatisé grâce à un script bootstrap.sh.
Ce script est injecté dans la VM via Terraform avec custom_data.

Il permet :

- d'installer Python et les dépendances
- de cloner le dépôt GitHub
- de créer le fichier .env
- de créer un service systemd
- de démarrer automatiquement l'application Flask

Connexion SSH à la VM :

```
→  terraform git:(main) ✗ ssh -i ~/.ssh/azure_vm_key azureuser@$(terraform ou
tput -raw vm_public_ip)

Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-azure x86_64)
```

Service Flask actif :

```
azureuser@flask-vm:~$ sudo systemctl status flaskapp --no-pager
● flaskapp.service - Flask Backend App
     Loaded: loaded (/etc/systemd/system/flaskapp.service; enabled; vendor pr
eset: enabled)
     Active: active (running) since Fri 2026-03-13 09:43:43 UTC; 3min 5s ago
   Main PID: 2964 (python3)
      Tasks: 1 (limit: 9524)
     Memory: 44.9M
        CPU: 453ms
     CGroup: /system.slice/flaskapp.service
             └─2964 /home/azureuser/Projet_Cloud_Computing/backend/venv/bin/…
```

Application accessible depuis la VM et l'éxterieur :

```
azureuser@flask-vm:~$ curl http://127.0.0.1:5001/
Flask API running with Azure Blob!
```

```
→  Desktop curl http://127.0.0.1:5001/
Flask API running with Azure Blob!%
```

---

7. Tests, problèmes rencontrés et conclusion :

Plusieurs tests ont été réalisés :

- démarrage du backend Flask
- vérification des endpoints CRUD
- upload d'un fichier vers Azure Blob Storage
- lecture des blobs
- création et suppression de l'infrastructure avec Terraform
- connexion SSH à la VM
- accès à l'application via l'IP publique

7.2 Problèmes rencontrés :

Plusieurs problèmes ont été rencontrés pendant le projet :

- certaines tailles de VM n'étaient pas disponibles dans la région France Central
- les IP publiques Basic n'étaient pas autorisées avec le compte Azure Student
- le port 5000 était déjà utilisé sur mac OS, ce qui a conduit à utiliser le port 5001

Erreur IP publique :

```
│ Error: updating Public I P Address (Subscription: "200abc54-63c9-4cb1-ba5b-
d57727df4338"
│ Resource Group Name: "terraform-flask-rg"
│ Public I P Addresses Name: "flask-public-ip"): performing CreateOrUpdate: u
nexpected status 400 (400 Bad Request) with error: IPv4BasicSkuPublicIpCountL
imitReached: Cannot create more than 0 IPv4 Basic SKU public IP addresses for
 this subscription in this region.
│
│   with azurerm_public_ip.vm_ip,
│   on main.tf line 35, in resource "azurerm_public_ip" "vm_ip":
│   35: resource "azurerm_public_ip" "vm_ip" {
│
```

7.3 Conclusion :

Ce projet a permis de mettre en pratique le développement d'une API Flask, l'utilisation d'Azure Blob Storage, l'automatisation d'infrastructure avec Terraform et le déploiement sur une machine virtuelle Azure.
Le résultat final est une application fonctionnelle, déployée automatiquement sur une VM et accessible via une IP publique.